

Sistemas numéricos

Capítulo 2 – Floyd

Sistemas de numeración

Números decimales:

La posición de cada dígito en un número decimal indica la magnitud de la cantidad representada y se le puede asignar un peso. Los pesos para los números enteros son las potencias positivas de diez, que aumentan de derecha a izquierda, comenzando por $10^0 = 1$. Para números fraccionarios, los pesos son las potencias negativas de diez que decrecen de izquierda a derecha comenzando por 10^{-1} .

Números binarios:

Los pesos de un número binario se basan en las potencias de dos. En general, con n bits se puede contar hasta un número decimal igual a

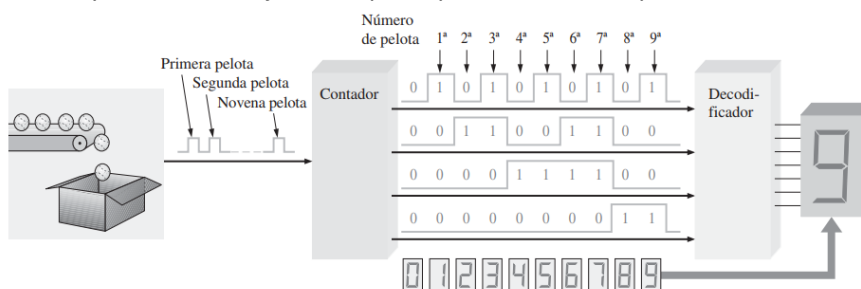
Número de decimales que se representan con n bits: $2^n - 1$.

El bit más a la derecha es el **LSB (Least Significant Bit, bit menos significativo)** en un número binario entero y tiene un peso de $2^0 = 1$. El bit más a la izquierda es el **MSB (Most Significant Bit, bit más significativo)**; su peso depende del tamaño del número. En un número binario con parte fraccionaria, el bit más a la izquierda es el MSB y tiene un peso de $2^{-1} = 0,5$. Los pesos fraccionarios de los respectivos bits decrecen de izquierda a derecha.

Utilidad:

- Los circuitos digitales que necesitan contar sucesos lo hacen en binarios. Puede tratarse de cualquier cosa, desde elementos que contar en una línea de montaje hasta operaciones de recuento en una computadora.
- Las computadoras utilizan los números binarios para determinar posiciones de memoria. Cada posición se asigna a un número unívoco denominado **dirección**.

Ejemplo de contador binario: El contador cuenta los pulsos procedentes de un sensor que detecta el paso de una pelota y genera una secuencia de **niveles lógicos (señales digitales)** en cada una de sus cuatro salidas paralelas. Cada conjunto de niveles lógicos representa un número binario de 4 bits (ALTO (H) = 1 y BAJO (L) = 0), como se indica. Cuando el decodificador recibe estas señales, decodifica cada conjunto de cuatro bits y lo convierte en el correspondiente número decimal en el display de 7 segmentos. Cuando el contador alcanza el estado binario 1001, es decir nueve pelotas, el display muestra el número 9 y una nueva caja se desplaza por la cinta transportadora.

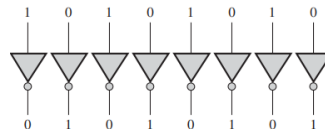


Binario a decimal: Suma los pesos de todos los 1s de un número binario para obtener el correspondiente valor decimal.

Decimal a binario: Para obtener el número binario correspondiente a un número decimal dado, divide el número decimal entre 2 hasta obtener un cociente igual a 0. Los restos forman el número binario. El primer resto es el (LSB) del número binario y el último resto es el (MSB). Los números decimales fraccionarios pueden convertirse en números binarios multiplicando sucesivamente por 2. Los dígitos acarreados o, acarreos, generados por las multiplicaciones **dan lugar al binario**.

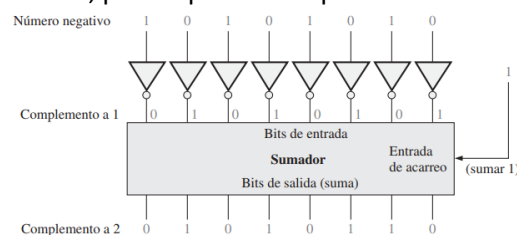
Complementos: permiten la representación de números negativos. La aritmética en complemento a 2 se usa comúnmente en las computadoras para manipular los números negativos.

- Complemento a uno: se halla cambiando todos los 1s por 0s y todos los 0s por 1s. La forma más sencilla de obtener el complemento a 1 de un número binario mediante un circuito digital es utilizando inversores en paralelo (circuitos NOT).



- Complemento a dos: El complemento a 2 de un número binario se obtiene sumando 1 al bit menos significativo del complemento a 1. El complemento a 2 de un número binario negativo puede obtenerse empleando inversores y un sumador.

Es importante notar que los números positivos tienen 0 en el MSB y los números negativos tienen un 1 en esta posición, por lo que el MSB puede ser visto como el bit de signo.



Números con signo: Existen tres formatos binarios para representar los números enteros con signo.

- *Signo – magnitud:* el bit más a la izquierda es el bit de signo y los restantes bits son los bits de magnitud. Para hallar el valor decimal hay que observar el primer bit. **Desventaja:** requiere dos pasos: sumar los pesos de los bits de magnitud y examinar el bit de signo para determinar si el número es positivo o negativo.
- *Complemento a uno:* un número negativo es el complemento a 1 del correspondiente número positivo. Para obtener el decimal correspondiente, se asigna el valor negativo al peso del bit de signo. Si da negativo, se suma 1 para obtener el resultado final. **Desventaja:** requiere añadir 1 a la suma de los pesos para los números negativos, pero no para los números positivos. También, son posibles dos representaciones del cero (00000000 y 11111111).
- *Complemento a dos:* Los números negativos son el complemento a 2 del correspondiente número positivo. Es el más importante. **Ventaja:** que requiere una adición de pesos independientemente de que el número sea positivo o negativo. Las computadoras utilizan el complemento a 2 para realizar operaciones aritméticas con números enteros negativos. La razón de ello es que la sustracción de un número es lo mismo que sumar el complemento a 2 del número.

Adición de números en complemento a 2: pueden producirse problemas cuando se suman dos números del mismo signo. Esto se resuelve utilizando overflow, es decir incorporando un bit más en el lugar del MSB para obtener los resultados correctos.

Rango de representación de binarios: El rango de magnitud de un número binario depende del número de bits (n). La fórmula para calcular el número de combinaciones diferentes de n bits es:

$$\text{Número total de combinaciones} = 2^n$$

Para los números con signo en complemento a 2, el rango de valores para números de n bits es:

$$\text{Rango} = -(2^{n-1}) \text{ hasta } +(2^{n-1}-1)$$

Números hexadecimales:

Consta de dieciséis caracteres y se usan fundamentalmente como una forma simplificada de representar o escribir los números binarios, ya que es muy fácil la conversión entre binario y hexadecimal.

Binario a hexadecimal: se parte el número binario en grupos de 4 bits, comenzando por el bit más a la derecha, y se reemplaza cada grupo de 4 bits por su símbolo hexadecimal equivalente.

Hexadecimal a binario: se reemplaza cada símbolo hexadecimal por el grupo de cuatro bits adecuado.

Hexadecimal a decimal: multiplicar el valor decimal de cada dígito hexadecimal por su peso, y luego realizar la suma de estos productos.

Decimal a hexadecimal: La división sucesiva por 16 de un número decimal generará el número hexadecimal equivalente formado por los restos de las divisiones.

Números octales:

Está formado por ocho dígitos, que son: 0, 1, 2, 3, 4, 5, 6, 7. Para contar por encima de 7, añadimos otra columna y continuamos.

De octal a decimal: multiplicando cada dígito por su peso (las potencias superiores de base 8 comenzando por 8^0) y sumando los productos.

Decimal a octal: usando la división sucesiva por 8, el número se conforma por los restos de esas divisiones.

Octal a binario: cada dígito octal se puede representar mediante un número binario de 3 dígitos.

Binario a octal: Para convertir a binario un número octal basta con reemplazar cada dígito octal con los tres bits apropiados.

Métodos generales para conversión entre decimales y otras bases:

La conversión de un número en base r a un decimal se realiza expandiendo el número en una serie de potencias y sumando todos los términos.

Para la operación inversa de convertir un número decimal a un número en base r . Se debe separar el procedimiento para la parte entera de un número y para la parte decimal. Para convertir un entero, se hace dividiendo el número en todos los cocientes de r y acumulando los restos. La conversión de una fracción decimal en base r se logra mediante un método similar al que se usa para enteros, excepto que se usa la multiplicación por r en vez de la división, y se acumulan enteros en vez de restos. El proceso de multiplicación de fracciones por r no termina necesariamente en cero, así que hay que decidir cuántos dígitos de la fracción convertida se usan.

Formato de representación de los números fraccionarios

Números en coma flotante:

Para representar números enteros muy grandes, son necesarios muchos bits. También surge un problema cuando se necesitan representar números con parte entera y parte fraccionaria. El sistema

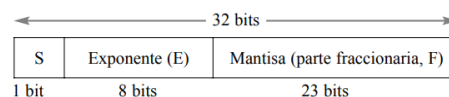
de **numeración en coma flotante** se basa en notación científica. Un número en coma flotante (también conocido como número real) tiene dos partes más un signo. La **mantisa** es la parte del número en coma flotante que representa la magnitud del número. El **exponente** es la parte de un número en coma flotante que representa el número de lugares que se va a desplazar el punto decimal (o punto binario).

Cuando el entero se expresa como un número en coma flotante, se normaliza desplazando el punto decimal a la izquierda de todos los dígitos, de modo que la mantisa es un número fraccionario y el exponente es una potencia de 10.

Ejemplo: Para 241.506.800, la mantisa es .2415068 y el exponente es 9. Este número en coma flotante se escribe: $0,2415068 \times 10^9$.

Utilidad: Además de la unidad central de procesamiento (CPU), las computadoras utilizan coprocesadores para realizar complejos cálculos matemáticos usando números en coma flotante. El propósito es aumentar el rendimiento liberando a la CPU para que pueda llevar a cabo otras tareas.

Números binarios en coma flotante de simple precisión: tienen 32 bits. Realmente, la mantisa consta de 24 bits, ya que, en cualquier número binario, el bit más a la izquierda (más significativo) es siempre un 1. Por tanto, este 1 se entiende que estará allí aunque no ocupe una posición de bit real.



Los 8 bits de los que consta el exponente representan un exponente desplazado que se ha obtenido mediante la adición de 127 al exponente real. El propósito de este desplazamiento es poder definir números muy grandes o muy pequeños sin necesidad de emplear un bit de signo diferente para el exponente. El exponente desplazado permite emplear un rango de valores para los exponentes comprendido entre -126 y $+128$.

Ejemplo: $1011010010001 = 1,011010010001 \times 2^{12}$. El primer 1 no va en la mantisa puesto que siempre existe un uno a la izquierda. El bit de signo es 0, el exponente se expresa $12+127=139$, y la parte fraccionaria será 1,011010010001; esto es:

S	E	F
0	10001011	0110100100010000000000

Capítulo 5.7 – Brown

Números con punto fijo:

Consta de partes entera y fraccionaria. Tienen un intervalo señalado por los dígitos significativos usados para representar el número. Por ejemplo, si se utilizan ocho dígitos y un signo para representar enteros decimales, entonces el intervalo de valores que pueden representarse va de 0 a 99999999. Si se emplean ocho dígitos para representar una fracción, entonces el intervalo representable va de 0.00000001 a 0,99999999.

Operaciones en punto fijo y flotante

Códigos BCD

Código decimal binario (BCD):

Las combinaciones binarias de un código de n bits se pueden determinar contando en binario de 0 a 2^n-1 . A cada elemento hay que asignarle una única combinación binaria, no está permitido que dos

elementos tengan el mismo valor; si no, el código es ambiguo. Un código binario tendrá algunas combinaciones binarias sin asignar si el número de elementos en un conjunto no es una potencia de 2. Los 10 dígitos decimales forman un conjunto así. Un código binario que distingue entre 10 elementos tiene que contener al menos cuatro bits, pero 6 de las 16 combinaciones posibles permanecerán sin asignar. Se pueden obtener numerosos códigos binarios diferentes colocando los cuatro bits con 10 combinaciones diferentes.

Código decimal binario significa que cada dígito decimal, de 0 hasta 9, se representa mediante un código binario de cuatro bits. Un número con n dígitos decimales requiere $4n$ bits en BCD.

Ventaja: los datos de entrada y salida se manejan por personas que usan el sistema decimal. Números BCD son números decimales y no binarios, aunque están representados con bits.

Desventajas: las reglas de la suma y la resta no son iguales a las sumas y restas decimales, se necesitan más bits para representar un número.

Código BCD a decimal: se comienza por el bit más a la derecha y se divide el código en grupos de cuatro bits. Después se escribe el dígito decimal representado por cada grupo de 4 bits.

Tipos de códigos BDC:

- 8421: La designación 8421 indica los pesos binarios de los cuatro bits (2^3 , 2^2 , 2^1 , 2^0). Las seis combinaciones que no se emplean (1010, 1011, 1100, 1101, 1110 y 1111) **no son válidas**.

Dígito decimal	0	1	2	3	4	5	6	7	8	9
Binario	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

Suma: si una suma de 4 bits es mayor que 9, o si genera un acarreo en el grupo de 4 bits, el resultado no es válido. En este caso, se suma 6 (0110) al grupo de 4 bits para saltar así los seis estados no válidos y pasar al código 8421.

- 2421: los pesos de los bits son 2^1 , 2^2 , 2^1 , 2^0 .
- 5421: los pesos son 5-4-2-1.
- exceso -3: para transformar decimales, lo que se hace es sumarle 3 unidades al número decimal, y luego convertir esto a binario.

Códigos alfanuméricos

Código ASCII:

American Standard Code for Information Interchange. Dispone de 128 caracteres que se representan mediante un código binario de 7 bits. Puede considerarse como un código de 8 bits en el que el MSB siempre es 0. Los primeros 32 caracteres son comandos no gráficos, sólo se utilizan para propósitos de control, como “nulo”, “avance de línea”, “inicio de texto” y “escape”. Los demás caracteres son símbolos gráficos que pueden mostrarse en pantalla, e incluyen las letras del alfabeto (mayúsculas y minúsculas), los diez dígitos decimales, los signos de puntuación y otros símbolos comúnmente utilizados.

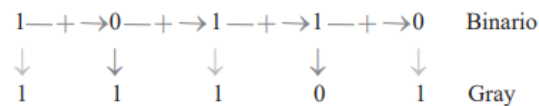
El **código ASCII extendido** posee 128 caracteres más, incluyendo caracteres alfabéticos no ingleses, símbolos de moneda no ingleses, letras griegas, símbolos matemáticos, caracteres para gráficos, caracteres para gráficos de barras y caracteres sombreados.

Código EBCDIC: viene de Extended Binary Coded Decimal Interchange Code - Código de intercambio decimal de código binario extendido. Representa caracteres alfanuméricos, controles y signos de puntuación. Cada carácter está compuesto por 8 bits = 1 byte, por eso EBCDIC define un total de 256 caracteres.

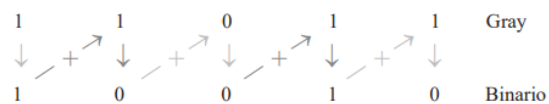
Código Gray:

Es un código sin pesos y no aritmético; es decir, no existen pesos específicos asignados a las posiciones de los bits. Solo varía un bit de un código al siguiente. Esta propiedad es importante en muchas **aplicaciones**, tales como los codificadores de eje de posición, en los que la susceptibilidad de error aumenta con el número de cambios de bit entre números adyacentes dentro de una secuencia.

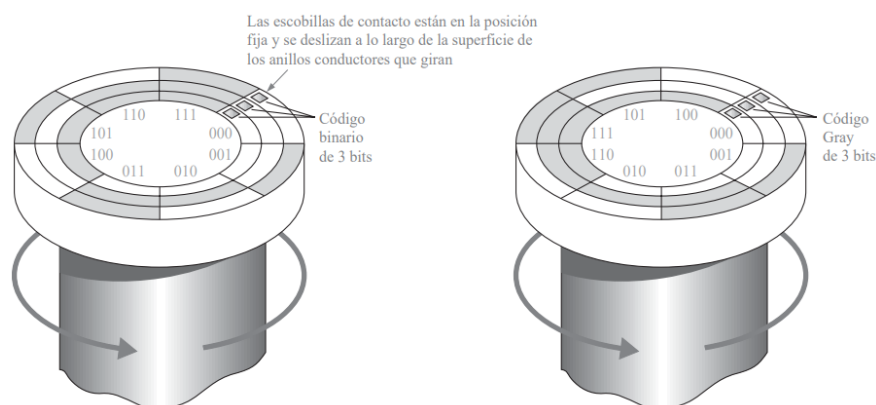
De binario a Gray: El bit más significativo (el que está más a la izquierda, MSB) en el código Gray es el mismo que el correspondiente MSB del número binario. Yendo de izquierda a derecha, sumar cada par adyacente de los bits en código binario para obtener el siguiente bit en código Gray. Los acarreos deben descartarse.



De Gray a binario: el bit más significativo (bit más a la izquierda) en el código binario es el mismo que el correspondiente bit en código Gray. A cada bit del código binario generado se le suma el bit en código Gray de la siguiente posición adyacente. Los acarreos se descartan.



Ejemplo: mecanismo codificador de eje de 3 bits. Cuando el eje gira en el sentido contrario a las agujas del reloj a través de los 360°, los ocho sectores se mueven y pasan por las tres escobillas produciendo una salida binaria de tres bits que indica la posición del eje. Puede producirse una salida errónea, por ejemplo pasando del 000 al 111 (codificado en código binario). Si la escobilla MSB se adelanta ligeramente, la posición se indicaría de forma incorrecta por una transición a 011 en lugar de 111 a 000. En este tipo de aplicaciones, es virtualmente imposible mantener el alineamiento mecánico preciso de todas las escobillas, por lo que siempre se producirán algunos errores en muchas de las transiciones entre sectores. Con código Gray, se asegura que sólo un bit variará entre sectores adyacentes. Esto significa que aunque las escobillas no puedan tener un alineamiento preciso, nunca se producirá un error de transición.



Códigos para la detección de errores

Capítulo 1 – Morris Mano

Método de paridad para la detección de errores: Un bit de paridad se añade al grupo de bits para hacer que el número total de 1s en el grupo sea siempre par o siempre impar, por esto el bit puede

ser de paridad par o paridad impar. Cuando el número de 1s no corresponda con el tipo de paridad, se ha detectado un error. Un bit de paridad facilita la detección de un único error de bit, pero no puede detectar dos errores dentro de un grupo. Usamos el bit extra en la posición más a la izquierda del código.

Desventaja: si se introduce un número par de cambios en los bits, no será detectado el error. Además, cuando los errores puedan ser detectados, no se puede saber la ubicación de tal, por lo que no podrán ser corregidos.

Utilidad: detectar errores en la comunicación y el procesamiento de datos.

Ejemplo: Asumiendo que se usa paridad par, se genera un bit de paridad par en el emisor para todos los caracteres ASCII de 7 bits. Se transmiten 8 bits, que incluyen los bits de paridad, hacia su destino. En el destino se comprueba la paridad de cada carácter: si la paridad de un carácter recibido no es par, significa que al menos un bit ha cambiado su valor durante la transmisión. Este método detecta sólo números impares de errores en cada carácter transmitido. Para número par de errores, se utilizan otros métodos.

Código 2/5: para un dato de 2 bits se agregan 5 bits de corrección que consisten en tres 0s y dos 1s. Esto añade 10 nuevas posibles combinaciones: la representación de los decimales de 0 a 9.

Ventaja: detecta todos los números impares de errores, algunos números pares de errores, y todos los errores de un solo bit.

Desventaja: no puede ubicar los errores.

Código de repetición: consiste en repetir múltiples veces cada bit de dato para asegurar que fue enviado correctamente. Por ejemplo, si tenemos un dato de un solo bit y utilizamos una repetición de $n=3$, el dato a enviar será de 3 bits idénticos. En caso de recibir un dato que no tenga sus tres bits idénticos, habrá un error. Es en realidad un código de Hamming con $r=2$, que verifica $2^2 \geq 2+1+1$ y $d=2^2-2-1=1$ cantidad de datos.

Desventaja: no corrige correctamente el error puesto que corrige una cantidad impar de bits. Si hubiera una cantidad par de cambios, interpreta que el dato original es uno erróneo. Además, aumentar la cantidad de bits de repetición NO aumenta la cantidad de bits que se pueden corregir.

Paridad Vertical y Longitudinal: La paridad horizontal y vertical es utilizada en algunos códigos de bloque para una combinación de chequeo de (LRC / VRC) para detectar errores. El proceso para calcular la paridad de bloque es el siguiente:

- Los caracteres a transmitir se agrupan en bloques de m filas y n columnas.
- Se calcula el bit de paridad de cada fila y se añade al final de la fila.
- Se calcula el bit de paridad de cada columna y se añade al final de la columna.

El bloque final a transmitir tendrá por tanto una fila y una columna más que el original. Adicionalmente se emplea un bit de paridad cruzada que se calcula a partir de los bits de paridad de filas y columnas.

Código de redundancia Cíclica (CRC): es un código de detección de errores usado frecuentemente en redes digitales y en dispositivos de almacenamiento para detectar cambios accidentales en los datos. Los bloques de datos ingresados en estos sistemas contienen un *valor de verificación adjunto* al final, basado en el residuo de una división de polinomios. El polinomio por el cual se divide el mensaje se denomina polinomio generador, y debe ser especificado cuando se utiliza este tipo de código.

El algoritmo utilizado por la verificación por redundancia cíclica es el siguiente:

- Se añaden r bits "0" a la derecha del mensaje (esto es, se añaden tantos ceros como grado tenga el polinomio generador).
- Se divide el polinomio obtenido por el polinomio generador. Esto equivale a aplicar una operación OR exclusiva (XOR) bit a bit.
- Se añade el resto de la división a la derecha del mensaje original.

La elección del polinomio generador es esencial si queremos detectar la mayoría de los errores que ocurran. Uno de los polinomios generadores que más se suelen utilizar es el estándar CCITT:

$$x^{16} + x^{12} + x^5 + 1$$

Cuando llega un mensaje, si éste y su CRC coinciden, puede asegurarse la integridad del dato.

Códigos para corrección de errores

El código Hamming de corrección de errores: debe incluirse más de un bit de paridad en un grupo de bits para poder corregir el error detectado, ya que se necesita más información, para identificar la posición del bit erróneo antes de poder corregirlo.

El código Hamming proporciona un método de corrección de un único bit erróneo. Si el número de bits de datos se designa por d, entonces el número de bits de paridad, p, se determina mediante la siguiente relación:

$$2^p \geq d + p + 1$$

A prueba y error se busca un número que satisfaga la ecuación. Los bits de paridad se sitúan en las posiciones que se han numerado haciéndolas corresponder con las potencias de dos en sentido ascendente (1, 2, 4, 8...). Los bits se enumeran en base 1. Para terminar, hay que asignar apropiadamente un valor de 1 o de 0 a cada uno de los bits de paridad. **Cada bit comprueba entonces todas las posiciones de bit, incluyéndose a sí mismo, que tienen un 1 en esa misma posición (posiciones expresadas en binario).** Los bits ubicados en estas posiciones serán los que hay que tener en cuenta para agregar cada bit de paridad.

Detectar posición del error:

1. Comience con el grupo comprobado por P1.
2. Compruebe si el grupo tiene la paridad correcta. Un 0 representa que la comprobación de paridad es correcta y un 1 que es incorrecta.
3. Repita el paso 2 para cada grupo de paridad.
4. El número binario formado por los resultados de todas las comprobaciones de paridad indica la posición del bit del código que es erróneo. Es el código de posición de error. La primera comprobación de paridad genera el bit menos significativo (LSB). Si todas las comprobaciones son correctas, no habrá error.

Extra:

Códigos auto complementarios: se dice que un código binario es auto complementario cuando el complemento a nueve del equivalente decimal de cualquier combinación del código puede hallarse invirtiendo los valores de cada uno de los bits (usando el operador de negación) y el resultado sigue siendo una combinación válida en ese código. Por ejemplo los códigos BCD